Taller 3

Maria Lucía Castillo García

Índice de contenido

Preguntas de reflexión....1
Evidencia de ejecución de cada actividad....3
Ejercicio 1....3
Ejercicio 2....3
Ejercicio 3....4
Ejercicio 4....4
Ejercicio 5....5
Ejercicio 6....5
Ejercicio 7....6
Ejercicio 8....6
Ejercicio 9....6
Ejercicio 10....7
Desafío final....7

Preguntas de reflexión

1) ¿Qué ventaja tienen los scripts frente a ejecutar comandos manualmente?

- Permiten automatizar tareas repetitivas, ahorrar tiempo y reducir errores humanos. Además, una vez escritos pueden reutilizarse muchas veces.
Ejemplo: si quisiera hacer un backup manualmente, tendría que escribir uno por uno los comandos `mkdir`, `tar`, `date`, y copiar el resultado a mano cada vez que quiera respaldar algo. Si un día se me llegara a olvidar alguno de los pasos o escribo mal el nombre de la carpeta, el backup queda mal hecho. En cambio, con el `backup.sh`, simplemente es correr el `backup.sh` y los mismos pasos, en el mismo orden correcto, se ejecutan solos cada vez.

2) ¿Cuándo utilizar un `for` y cuándo un `while`?

- `for`: cuando se conoce el conjunto de elementos a recorrer (por ejemplo, todos los archivos .txt).
`while`: cuando se repite una acción mientras una condición sea verdadera.

3) ¿Qué representa `$?`?

- Representa el código de salida del último comando ejecutado. Un valor de 0 indica éxito y otros valores indican algún error.

4) ¿Por qué es importante validar la existencia de archivos antes de manipularlos?

- Porque evita errores al intentar copiar, mover o eliminar archivos o directorios que no existen, haciendo los scripts más robustos y seguros. Esto se relaciona con el uso de condicionales para verificar condiciones antes de ejecutar acciones.
Ejemplo: si en el `backup.sh` no se hubiese puesto el `if [ -d "$directorio" ]` al inicio, y alguien escribe mal el nombre de la carpeta (por ejemplo "prueva" en vez de "prueba"), el `tar` intentaría comprimir algo que no existe y el script tronaría con un error feo, o peor, generaría un archivo .tar.gz vacío sin que te des cuenta.

5) ¿Cómo podrían integrarse estos scripts en procesos de automatización para servidores Linux, servicios en la nube o dispositivos IoT?

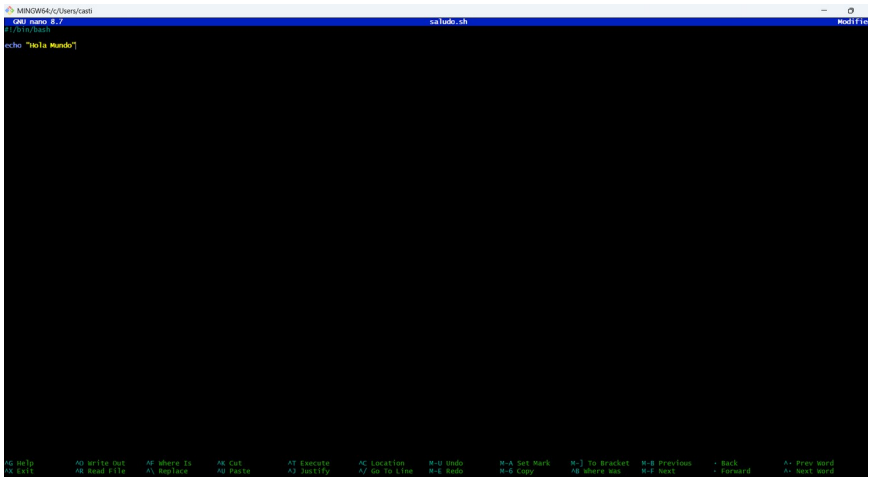
- Los scripts pueden programarse para realizar respaldos automáticos, monitorear sistemas, procesar archivos o ejecutar tareas de mantenimiento sin intervención manual. De esta manera ayudan a automatizar procesos en servidores, plataformas en la nube y dispositivos IoT.
Ejemplo: el `backup.sh`, tal como está, podría programarse con una herramienta llamada cron (el "programador de tareas" de Linux) para que se ejecute solo, por ejemplo, todas las noches a las 2am en un servidor; sin que nadie tenga que conectarse manualmente a correrlo. En un dispositivo IoT (como un sensor que guarda datos de temperatura), un script parecido podría revisar cada hora si hay archivos nuevos de datos y respaldarlos automáticamente a la nube, usando exactamente la misma lógica de variables + if + tar.

Evidencia de ejecución de cada actividad

Ejercicio 1

saludo.sh

```
casti@MLC MINGW64 ~
$ nano saludo.sh
```




```
casti@MLC MINGW64 ~
$ nano saludo.sh

casti@MLC MINGW64 ~
$ chmod +x saludo.sh

casti@MLC MINGW64 ~
$ ./saludo.sh
Hola Mundo

casti@MLC MINGW64 ~
$
```

Ejercicio 2

Variables

```
casti@MLC MINGW64 ~
$ nano variables.sh

casti@MLC MINGW64 ~
$ chmod +x variables.sh

casti@MLC MINGW64 ~
$ ./variables.sh
Hola Juan

casti@MLC MINGW64 ~
$
```

```
#!/bin/bash

nombre="Juan"
echo "Hola $nombre"
```

Ejercicio 3

Entrada con read

```
casti@MLC MINGW64 ~
$ nano entrada.sh
```

```
 GNU nano 8.7
#!/bin/bash

read -p "Ingrese su nombre: " nombre
echo "Bienvenido $nombre"
```

```
casti@MLC MINGW64 ~
$ chmod +x entrada.sh

casti@MLC MINGW64 ~
$ ./entrada.sh
Ingrese su nombre: Lucia
Bienvenido Lucia

casti@MLC MINGW64 ~
$
```

Ejercicio 4

Condicional

```
casti@MLC MINGW64 ~
$ nano condicional.sh
```

```
 GNU nano 8.7
#!/bin/bash

if [ -f archivo.txt ]
then
        echo "Existe"
else
        echo "No existe"
fi
```

```
casti@MLC MINGW64 ~
$ chmod +x condicional.sh

casti@MLC MINGW64 ~
$ ./condicional.sh
No existe

casti@MLC MINGW64 ~
```

Ejercicio 5

For

```
casti@MLC MINGW64 ~
$ nano for.sh
```

```
MINGW64:/c/Users/casti
  GNU nano 8.7
#!/bin/bash

for i in {1..10}
do
        echo $i
done
```

```
casti@MLC MINGW64 ~
$ chmod +x for.sh

casti@MLC MINGW64 ~
$ ./for.sh
1
2
3
4
5
6
7
8
9
10
```

Ejercicio 6

While

```
casti@MLC MINGW64 ~
$ nano while.sh
```

```
#!/bin/bash

contador=1

while [ $contador -le 5 ]
do
        echo $contador
        contador=$((contador+1))
done
```

```
casti@MLC MINGW64 ~
$ chmod +x while.sh

casti@MLC MINGW64 ~
$ ./while.sh
1
2
3
4
5
```

Ejercicio 7

Funciones

```
casti@MLC MINGW64 ~
$ nano funciones.sh
```

```
#!/bin/bash

saludar(){
        echo "Hola"

}

saludar
```

```
casti@MLC MINGW64 ~
$ chmod +x funciones.sh

casti@MLC MINGW64 ~
$ ./funciones.sh
Hola
```

Ejercicio 8

Parámetros

```
casti@MLC MINGW64 ~
$ nano script.sh
```

```
#!/bin/bash

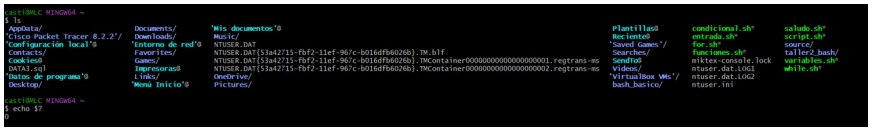
echo "Primer parámetro: $1"
echo "Segundo parámetro: $2"
```

```
casti@MLC MINGW64 ~
$ chmod +x script.sh

casti@MLC MINGW64 ~
$ ./script.sh Juan Pérez
Primer parámetro: Juan
Segundo parámetro: Pérez
```

Ejercicio 9

Código de salida



```
casti@MLC MINGW64 ~
$ ls inexistente
ls: cannot access 'inexistente': No such file or directory

casti@MLC MINGW64 ~
$ echo $?
2

casti@MLC MINGW64 ~
$
```

Ejercicio 10

Proyecto integrador: proyecto.sh

```
casti@MLC MINGW64 ~
$ nano proyecto.sh
```

```
#!/bin/bash

fecha=$(date +%Y-%m-%d)

mkdir -p "$fecha"

cantidad=0

for archivo in *.txt
do
        cp "$archivo" "$fecha"
        if [ $? -eq 0 ]
        then
                cantidad=$((cantidad+1))
        fi
done

echo "Se copiaron $cantidad archivos .txt" > "$fecha/reporte.log"

if [ $cantidad -gt 0 ]
then
        echo "Éxito: se copiaron $cantidad de archivos"
else
        echo "Error: no se copió ningún archivo"

fi
```

```
casti@MLC MINGW64 ~
$ chmod +x proyecto.sh

casti@MLC MINGW64 ~
$ ./proyecto.sh
Éxito: se copiaron 3 de archivos
```

Desafío final

Creación de backup.sh

```
casti@MLC MINGW64 ~
$ nano backup.sh
```

```
#!/bin/bash

read -p "Ingrese el nombre del directorio: " directorio

if [ -d "$directorio" ]
then
        echo "El directorio existe"

        mkdir -p backup
        fecha=$(date +%Y-%m-%d)
        tar -czf backup/respaldo_$fecha.tar.gz "$directorio"

        if [ $? -eq 0 ]
        then
                estado="Existoso"
        else
                estado="Fallido"
        fi

        cantidad=$(find "$directorio" -type f | wc -l)
        tamano=$(du -h backup/respaldo_$fecha.tar.gz | cut -f1)

        echo "$(date): Directorio='$directorio' | Archivos=$cantidad | Tamaño=$tamano |
        Estado=$estado" >> backup/backup.log

        echo "-------- RESUMEN ---------"
        echo "Fecha: $fecha"
        echo "Cantidad de archivos respaldados: $cantidad"
        echo "Tamaño del respaldo: $tamano"
        echo "Estado del proceso: $estado"
else
        echo "El directorio no existe"
fi
```

Ejecución

```
casti@MLC MINGW64 ~
$ chmod +x backup.sh

casti@MLC MINGW64 ~
$ mkdir prueba

casti@MLC MINGW64 ~
$ touch prueba/archivo1.txt prueba/archivo2.txt

casti@MLC MINGW64 ~
$ ./backup.sh
Ingrese el nombre del directorio: prueba
El directorio existe
-------- RESUMEN ---------
Fecha: 2026-08-09
Cantidad de archivos respaldados: 2
Tamaño del respaldo: 1.0K
Estado del proceso: Existoso
```

Carpeta backup y backup.log

```
casti@MLC MINGW64 ~
$ ls backup
respaldo_2026-08-09.tar.gz
```

```
casti@MLC MINGW64 ~
$ cat backup/backup.log
Sun Aug  9 16:32:19 HPS 2026: Directorio='prueba' | Archivos=2 | Tamaño=1.0K |
        Estado=Existoso
```